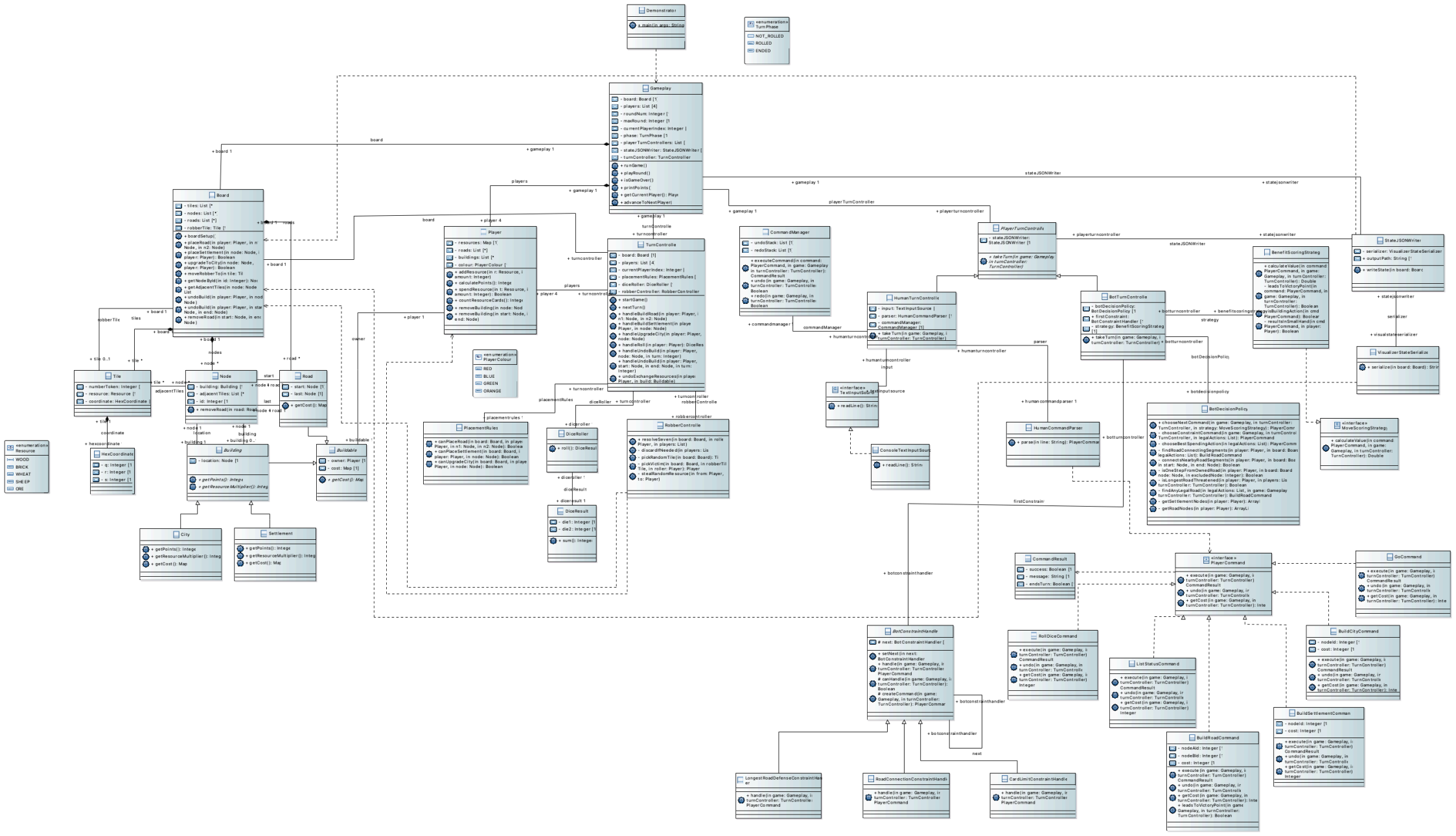
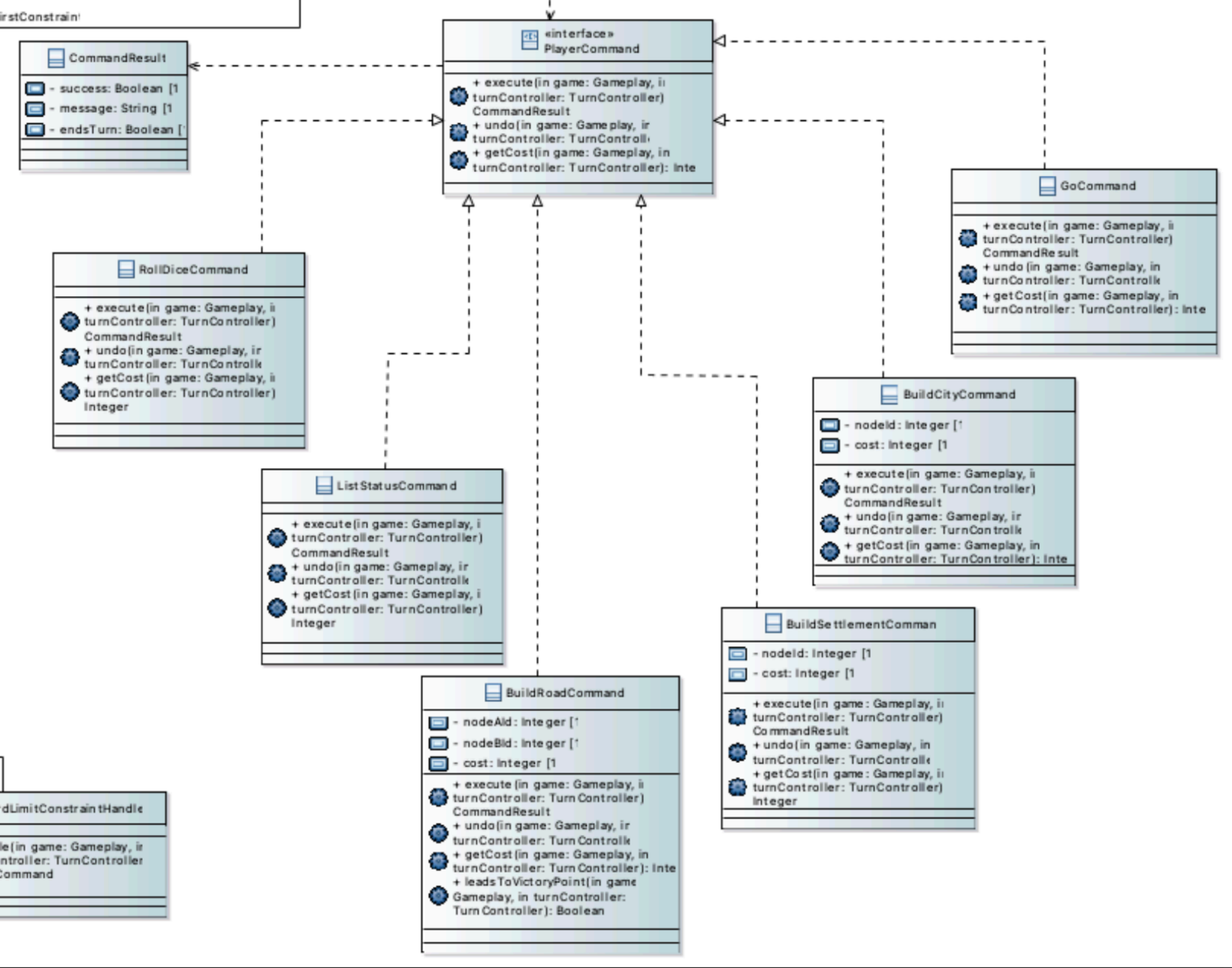
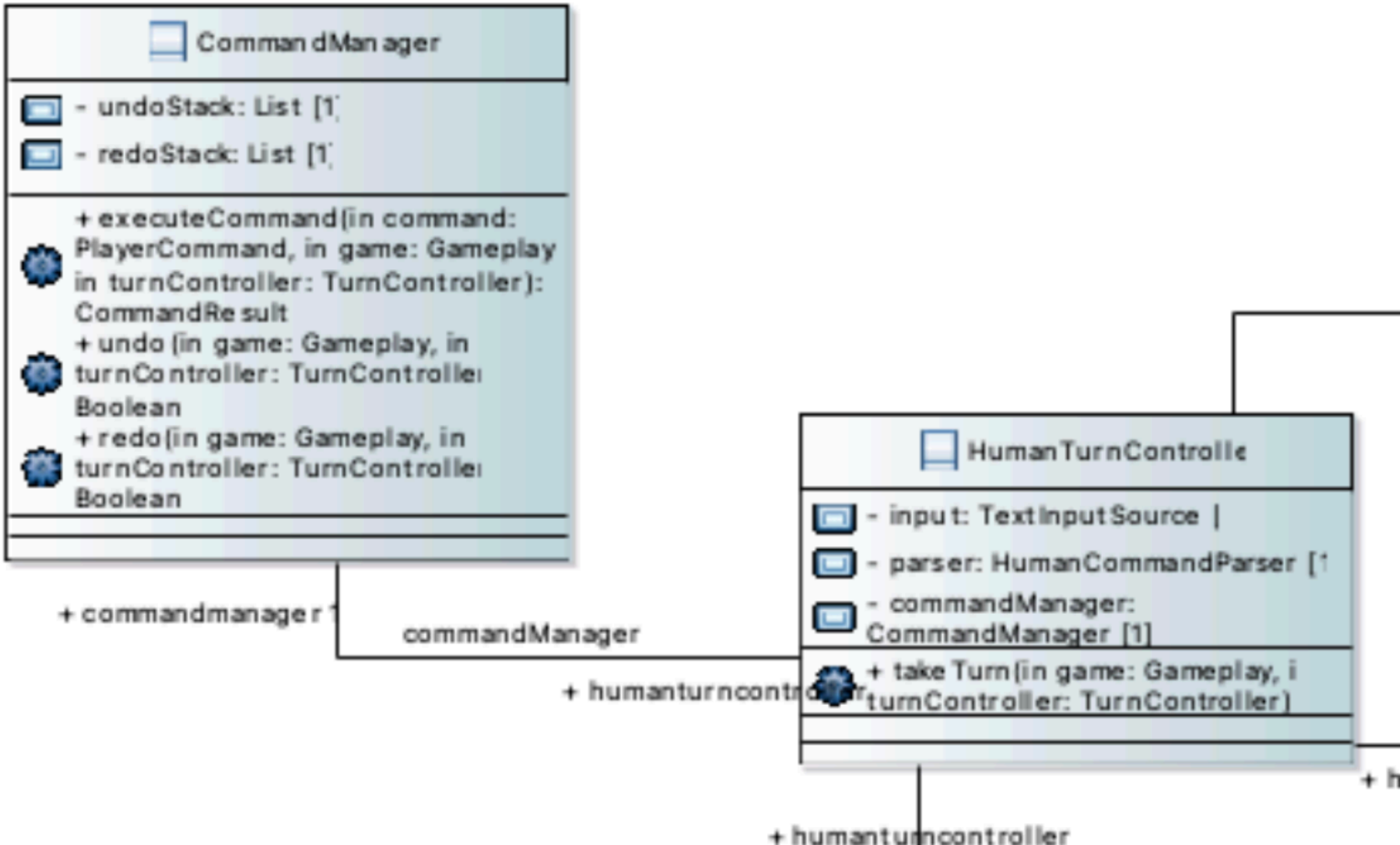
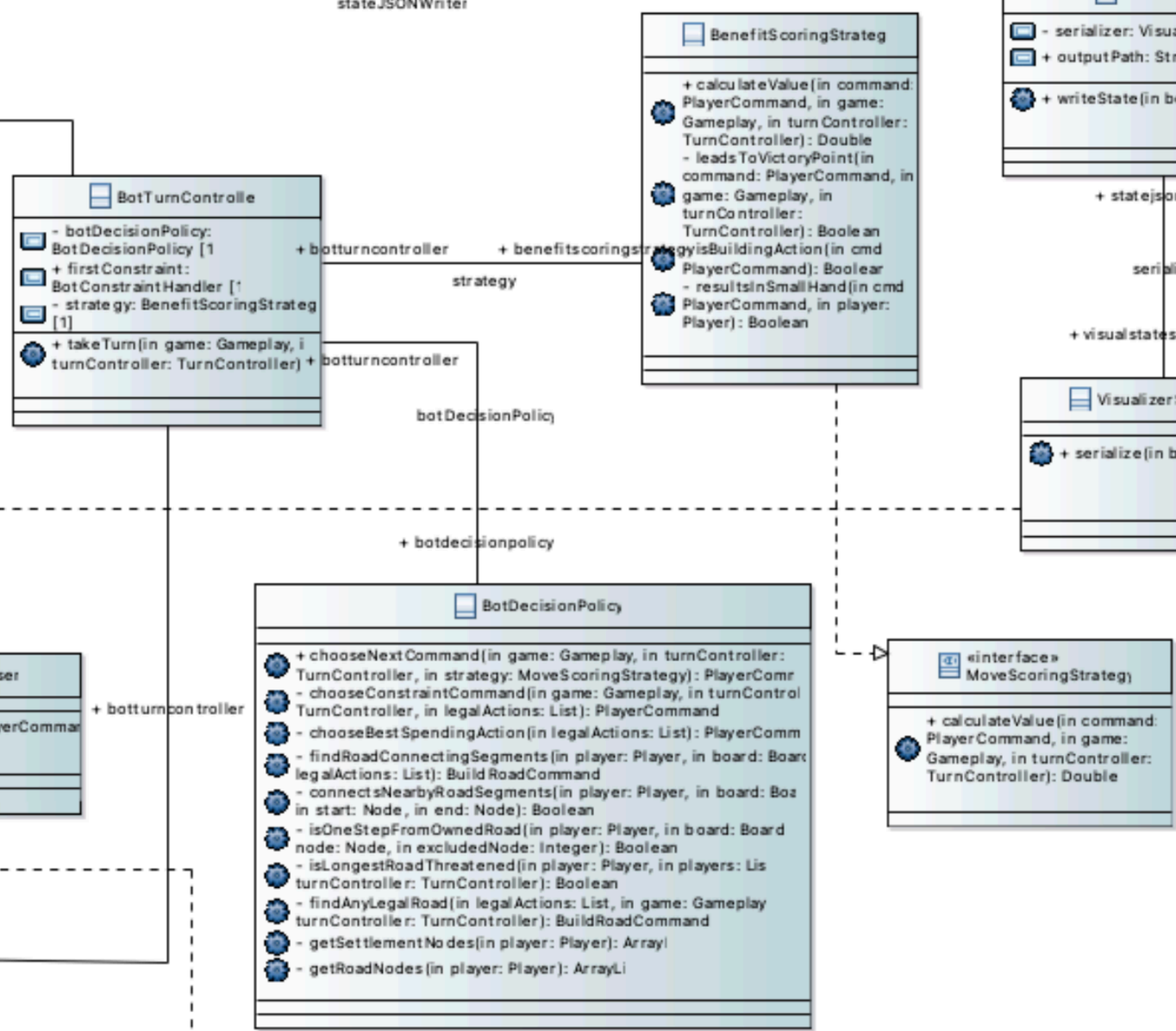
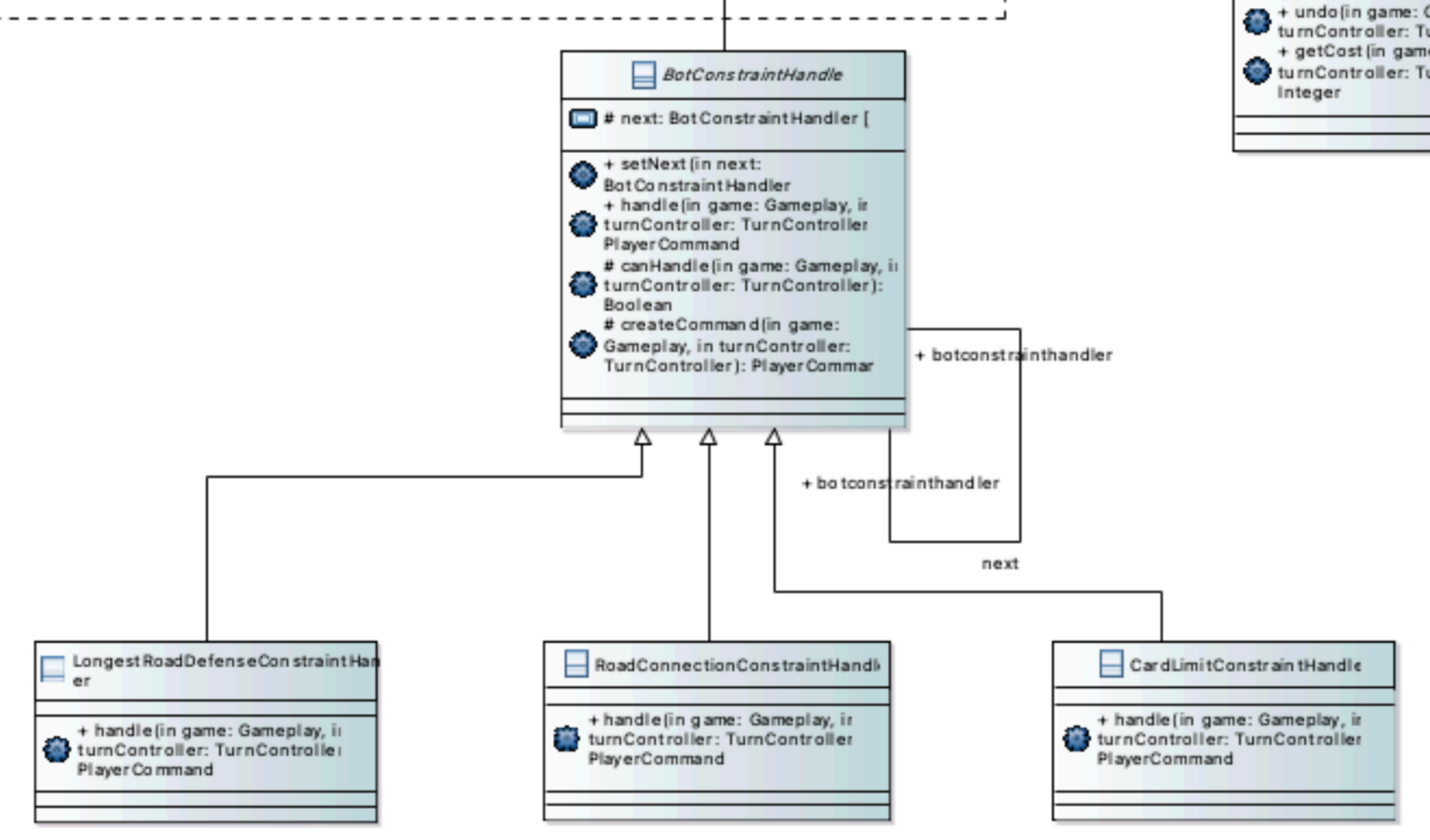

Demonstrator
+ main(in args: String
«enumeration» TurnPhase
NOT_ROLLED
ROLLED
ENDED
Gameplay
- board: Board [1]
- players: List [4]
- roundNum: Integer [
- maxRound: Integer [1
- currentPlayerIndex: Integer [
- phase: TurnPhase [1]
- playerTurnControllers: List [
- stateJSONWriter: StateJSONWriter [
- turnController: TurnController
+ runGame()
+ playRound()
+ isGameOver()
+ printPoints(
+ getCurrentPlayer(): Playe
+ advanceToNextPlayer(
Board
- tiles: List [*
- nodes: List [*
- roads: List [*]
- robberTile: Tile [1
+ boardSetup(
+ placeRoad(in player: Player, in n Node, in n2: Node)
+ placeSettlement(in node: Node, i player: Player): Boolean
+ upgradeToCity(in node: Node, player: Player): Boolean
+ moveRobberTo(in tile: Til
+ getNodeById(in id: Integer): Nod
+ getAdjacentTiles(in node: Node List
+ undoBuild(in player: Player, in nod Node)
+ undoBuild(in player: Player, in sta Node, in end: Node)
+ removeRoad(in start: Node, in end Node)
Player
- resources: Map [1
- roads: List [*]
- buildings: List [*
- colour: PlayerColour [
+ addResource(in r: Resource, i amount: Integer)
+ calculatePoints(): Intege
+ spendResource(in t: Resource, i amount: Integer): Boolean
+ countResourceCards(): Intege
+ removeBuilding(in node: Nod
+ removeBuilding(in start: Node, i end: Node)
TurnController
- board: Board [1]
- players: List [4
- currentPlayerIndex: Integer [
- placementRules: PlacementRules [
- diceRoller: DiceRoller [
- robberController: RobberController
+ startGame()
+ nextTurn()
+ handleBuildRoad(in player: Player, i n1: Node, in n2: Node)
+ handleBuildSettlement(in playe Player, in node: Node)
+ handleUpgradeCity(in player: Player, node: Node)
+ handleRoll(in player: Player): DiceRes
+ handleUndoBuild(in player: Player, node: Node, in turn: Integer)
+ handleUndoBuild(in player: Player, start: Node, in end: Node, in turn: Integer)
+ undoExchangeResources(in playe Player, in build: Buildable)
CommandManager
- undoStack: List [1
- redoStack: List [1
+ executeCommand(in command: PlayerCommand, in game: Gameplay in turnController: TurnController): CommandResult
+ undo(in game: Gameplay, in turnController: TurnController: Boolean
+ redo(in game: Gameplay, in turnController: TurnController: Boolean
PlayerTurnControls
- stateJSONWriter: StateJSONWriter [1
+ takeTurn(in game: Gameplay, in turnController: TurnController)
StateJSONWriter
- serializer: VisualizerStateSerializer
- outputPath: String [1
+ writeState(in board: Board
BenefitScoringStrateg
+ calculateValue(in command: PlayerCommand, in game: Gameplay, in turnController: TurnController): Double
- leadsToVictoryPoint(in command: PlayerCommand, in game: Gameplay, in turnController: TurnController): Boolean
- applyBuildingAction(in cmd PlayerCommand): Boolear
- resultsinSmallHand(in cmd PlayerCommand, in player: Player): Boolean
HumanTurnControlle
- input: TextInputSource [
- parser: HumanCommandParser [1
- commandManager: CommandManager [1]
+ takeTurn(in game: Gameplay, i turnController: TurnController)
BotTurnControlle
- botDecisionPolicy: BotDecisionPolicy [1
+ firstConstraint: BotConstraintHandler [
- strategy: BenefitScoringStrateg [1]
+ takeTurn(in game: Gameplay, i turnController: TurnController)
VisualizerStateSerialize
+ serialize(in board: Board): Strin
«interface» TextInputSource
+ readLine(): Strin
ConsoleTextInputSourc
+ readLine(): Strin
HumanCommandParser
+ parse(in line: String): PlayerComma
BotDecisionPolicy
+ chooseNextCommand(in game: Gameplay, in turnController: TurnController, in strategy: MoveScoringStrategy): PlayerComr
- chooseConstructCommand(in game: Gameplay, in turnContro TurnController, in legalActions: List): PlayerCommand
- chooseBestSpendingAction(in legalActions: List): PlayerComm
- findRoadConnectingSegments(in player: Player, in board: Boar legalActions: List): BuildRoadCommand
- connectNearbyRoadSegments(in player: Player, in board: Boa in start: Node, in end: Node): Boolean
- isOneStepFromOwnedRoad(in player: Player, in board: Board node: Node, in excludedNode: Integer): Boolean
- isLongestRoadThreatened(in player: Player, in players: Lis turnController: TurnController): Boolean
- findAnyLegalRoad(in legalActions: List, in game: Gameplay turnController: TurnController): BuildRoadCommand
- getSettlementNodes(in player: Player): Array:
- getRoadNodes(in player: Player): ArrayLi
«interface» MoveScoringStrategy
+ calculateValue(in command: PlayerCommand, in game: Gameplay, in turnController: TurnController): Double
«enumeration» PlayerColour
RED
BLUE
GREEN
ORANGE
«enumeration» Resource
WOOD
BRICK
WHEAT
SHEEP
ORE
Tile
- numberToken: Integer [
- resource: Resource [
- coordinate: HexCoordinate
Node
- building: Building [
- adjacentTiles: List [*
- id: Integer [1
+ removeRoad(in road: Road
Road
- start: Node [1]
- last: Node [1]
+ getCost(): Map
HexCoordinate
- q: Integer [1]
- r: Integer [1]
- s: Integer [1]
Building
- location: Node [1
+ getPoints(): Intege
+ getResourceMultiplier(): Integ
Buildable
- owner: Player [1]
- cost: Map [1]
+ getCost(): Map
City
+ getPoints(): Intege
+ getResourceMultiplier(): Integ
+ getCost(): Map
Settlement
+ getPoints(): Intege
+ getResourceMultiplier(): Integ
+ getCost(): Map
PlacementRules
+ canPlaceRoad(in board: Board, in playe Player, in n1: Node, in n2: Node): Boolea
+ canPlaceSettlement(in board: Board, i player: Player, in node: Node): Boolean
+ canUpgradeCity(in board: Board, in playe Player, in node: Node): Boolean
DiceRoller
+ roll(): DiceResul
DiceResult
- die1: Integer [1]
- die2: Integer [1]
+ sum(): Integer
RobberController
+ resolveSeven(in board: Board, in rolle Player, in players: List)
- discardIfNeeded(in players: Lis
- pickRandomTile(in board: Board): Ti
- pickVictim(in board: Board, in robberTil Tile, in roller: Player): Player
- stealRandomResource(in from: Player, to: Player)
CommandResult
- success: Boolean [1
- message: String [1
- endsTurn: Boolean [
«interface» PlayerCommand
+ execute(in game: Gameplay, i turnController: TurnController) CommandResult
+ undo(in game: Gameplay, ir turnController: TurnControlle
+ getCost(in game: Gameplay, in turnController: TurnController): Inte
GoCommand
+ execute(in game: Gameplay, i turnController: TurnController) CommandResult
+ undo(in game: Gameplay, in turnController: TurnControlle
+ getCost(in game: Gameplay, in turnController: TurnController): Inte
BotConstraintHandle
# next: BotConstraintHandler [
+ setNext(in next: BotConstraintHandler
+ handle(in game: Gameplay, ir turnController: TurnControlle PlayerCommand
# canHandle(in game: Gameplay, i turnController: TurnController): Boolean
# createCommand(in game: Gameplay, in turnController: TurnController): PlayerCommar
RollDiceCommand
+ execute(in game: Gameplay, i turnController: TurnController) CommandResult
+ undo(in game: Gameplay, in turnController: TurnControlle
+ getCost(in game: Gameplay, i turnController: TurnController) Integer
ListStatusCommand
+ execute(in game: Gameplay, i turnController: TurnController) CommandResult
+ undo(in game: Gameplay, ir turnController: TurnControlle
+ getCost(in game: Gameplay, i turnController: TurnController) Integer
BuildCityCommand
- nodeId: Integer [1
- cost: Integer [1
+ execute(in game: Gameplay, i turnController: TurnController) CommandResult
+ undo(in game: Gameplay, ir turnController: TurnControlle
+ getCost(in game: Gameplay, in turnController: TurnController): Inte
BuildSettlementComman
- nodeId: Integer [1
- cost: Integer [1
+ execute(in game: Gameplay, ii turnController: TurnController) CommandResult
+ undo(in game: Gameplay, in turnController: TurnControlle
+ getCost(in game: Gameplay, ii turnController: TurnController) Integer
BuildRoadCommand
- nodeAId: Integer [1
- nodeBId: Integer [1
- cost: Integer [1
+ execute(in game: Gameplay, ii turnController: TurnController) CommandResult
+ undo(in game: Gameplay, ir turnController: TurnControlle
+ getCost(in game: Gameplay, in turnController: TurnController): Inte
+ leadsToVictoryPoint(in game: Gameplay, in turnController: TurnController): Boolean
LongestRoadDefenseConstraintHan er
+ handle(in game: Gameplay, ii turnController: TurnControlle PlayerCommand
RoadConnectionConstraintHandl
+ handle(in game: Gameplay, ir turnController: TurnControlle PlayerCommand
CardLimitConstraintHandle
+ handle(in game: Gameplay, ir turnController: TurnControlle PlayerCommand
board
+ board 1
+ gameplay 1
players
+ player 4
+ player 1
turncontroller
+ turncontroller
+ playerturncontroller
stateJSONWriter
+ statejsonwriter
playerTurnController
+ botdecisionpolicy
+ botconstraint handler
+ botconstraint handler
next
firstConstrain
parser
+ humancommandparser 1
+ humanturncontroller
+ botturncontroller
botDecisionPolicy
strategy
+ botturncontroller
+ benefitscoringst
+ visualizerstateserializer
serializer
+ statejsonwriter
+ humanturncontroller
input
+ humanturncontroller
commandManager
+ commandmanager
robberTile
tiles
+ tile 0..1
+ tile *
+ board 1
nodes
+ node *
+ road *
adjacentTiles
+ tile *
+ node
coordinate
+ hexcoordinate
+ tile 1
location
+ node 1
building
+ building 0..
+ buildable
owner
placementRules
+ placementrules 1
+ turncontroller
dice Roller
+ turncontroller
robberControlle
+ robbercontroller
+ dicerolle r
+ dicerolle r
dice Result
+ diceresult 1
start
last
+ node 4 road
+ node 4 road 1


irstConstrain
CommandResult
- success: Boolean [1
- message: String [1
- endsTurn: Boolean [
«interface»
PlayerCommand
+ execute(in game: Gameplay, i
turnController: TurnController)
CommandResult
+ undo(in game: Gameplay, ir
turnController: TurnControll
+ getCost(in game: Gameplay, in
turnController: TurnController): Inte
GoCommand
+ execute(in game: Gameplay, i
turnController: TurnController)
CommandResult
+ undo(in game: Gameplay, in
turnController: TurnControll
+ getCost(in game: Gameplay, in
turnController: TurnController): Inte
RollDiceCommand
+ execute(in game: Gameplay, i
turnController: TurnController)
CommandResult
+ undo(in game: Gameplay, ir
turnController: TurnControll
+ getCost(in game: Gameplay, i
turnController: TurnController)
Integer
BuildCityCommand
- nodeId: Integer [1
- cost: Integer [1
+ execute(in game: Gameplay, i
turnController: TurnController)
CommandResult
+ undo(in game: Gameplay, ir
turnController: TurnControll
+ getCost(in game: Gameplay, in
turnController: TurnController): Inte
ListStatusCommand
+ execute(in game: Gameplay, i
turnController: TurnController)
CommandResult
+ undo(in game: Gameplay, ir
turnController: TurnControll
+ getCost(in game: Gameplay, i
turnController: TurnController)
Integer
BuildSettlementComman
- nodeId: Integer [1
- cost: Integer [1
+ execute(in game: Gameplay, i
turnController: TurnController)
CommandResult
+ undo(in game: Gameplay, in
turnController: TurnControll
+ getCost(in game: Gameplay, i
turnController: TurnController)
Integer
BuildRoadCommand
- nodeAId: Integer [1
- nodeBId: Integer [1
- cost: Integer [1
+ execute(in game: Gameplay, i
turnController: TurnController)
CommandResult
+ undo(in game: Gameplay, ir
turnController: TurnControll
+ getCost(in game: Gameplay, in
turnController: TurnController): Inte
+ leadsToVictoryPoint(in game
Gameplay, in turnController:
TurnController): Boolean
dLimitConstraintHandle
le(in game: Gameplay, ir
ntroller: TurnController
Command


CommandManager
- undoStack: List [1
- redoStack: List [1
+ executeCommand(in command: PlayerCommand, in game: Gameplay in turnController: TurnController): CommandResult
+ undo (in game: Gameplay, in turnController: TurnController: Boolean
+ redo(in game: Gameplay, in turnController: TurnController: Boolean
+ commandmanager
commandManager
HumanTurnControlle
- input: TextInputSource |
- parser: HumanCommandParser [1
- commandManager: CommandManager [1]
+ takeTurn(in game: Gameplay, i turnController: TurnController)
+ humanturncontroller


stateJSONWriter
BenefitScoringStrateg
+ calculateValue(in command: PlayerCommand, in game: Gameplay, in turnController: TurnController): Double
- leadsToVictoryPoint(in command: PlayerCommand, in game: Gameplay, in turnController: TurnController): Boolean
- isBuildingAction(in cmd PlayerCommand): Boolear
- resultsInSmallHand(in cmd PlayerCommand, in player: Player): Boolean
- serializer: Visua
+ outputPath: Str
+ writeState(in b
+ statejson
serial
+ visualstates
Visualizer
+ serialize(in b
BotTurnControlle
- botDecisionPolicy: BotDecisionPolicy [1
+ firstConstraint: BotConstraintHandler [1
- strategy: BenefitScoringStrateg [1]
+ takeTurn(in game: Gameplay, i turnController: TurnController)
+ botturncontroller
+ benefitscoringstrategy
strategy
+ botturncontroller
botDecisionPolicy
+ botdecisionpolicy
BotDecisionPolicy
+ chooseNextCommand(in game: Gameplay, in turnController: TurnController, in strategy: MoveScoringStrategy): PlayerComr
- chooseConstraintCommand(in game: Gameplay, in turnControl TurnController, in legalActions: List): PlayerCommand
- chooseBestSpendingAction(in legalActions: List): PlayerComm
- findRoadConnectingSegments(in player: Player, in board: Boar legalActions: List): BuildRoadCommand
- connectsNearbyRoadSegments(in player: Player, in board: Boa in start: Node, in end: Node): Boolean
- isOneStepFromOwnedRoad(in player: Player, in board: Board node: Node, in excludedNode: Integer): Boolean
- isLongestRoadThreatened(in player: Player, in players: Lis turnController: TurnController): Boolean
- findAnyLegalRoad(in legalActions: List, in game: Gameplay turnController: TurnController): BuildRoadCommand
- getSettlementNodes(in player: Player): ArrayL
- getRoadNodes(in player: Player): ArrayLi
+ botturncontroller
«interface»
MoveScoringStrateg
+ calculateValue(in command: PlayerCommand, in game: Gameplay, in turnController: TurnController): Double


BotConstraintHandle
# next: BotConstraintHandler [
+ setNext (in next: BotConstraintHandler
+ handle (in game: Gameplay, ir turnController: TurnController PlayerCommand
# canHandle (in game: Gameplay, ii turnController: TurnController): Boolean
# createCommand (in game: Gameplay, in turnController: TurnController): PlayerCommar
+ botconstrainthandler
+ botconstrainthandler
next
+ undo (in game: C turnController: Tu
+ getCost (in game turnController: Tu Integer
LongestRoadDefenseConstraintHan er
+ handle (in game: Gameplay, ii turnController: TurnControlle PlayerCommand
RoadConnectionConstraintHandl
+ handle (in game: Gameplay, ir turnController: TurnController PlayerCommand
CardLimitConstraintHandle
+ handle (in game: Gameplay, ir turnController: TurnController PlayerCommand